

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное
учреждение высшего образования
«Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовой работе
по дисциплине «Теория графов»
Реализация экспертной системы:
«Выбор животного на основе заданных характеристик»

Студент,
группа 5130201/40001

_____ Ови Мд Шамин Ясир

Преподаватель,

_____ Востров Алексей Владимирович

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Предметная область	4
2 Математическое описание	6
2.1 Экспертная система и её особенности	6
2.2 Продукционная модель представления знаний	7
2.3 Бинарное дерево решений	7
3 Особенности реализации	10
3.1 Общая функция задания вопроса и проверки ввода	10
3.2 Функция для вопросов типа да/нет	10
3.3 Правило вывода рекомендации	10
3.4 Правило перезапуска	11
3.5 Правила дерева решений	11
3.6 Правила-листья (рекомендации)	12
4 Результаты работы программы	13
4.1 Пример №1: путь yes-yes-yes-yes, рекомендация — Dog	13
4.2 Пример №2: повторный сеанс и завершение работы	13
4.3 Пример №3: обработка некорректного ввода	14
Список использованной литературы	16
Приложение А. Файл pet_expert.clp	17

Введение

Экспертные системы — системы, основанные на моделях знаний из определённых предметных областей.

Экспертная система включает в себя знания высококвалифицированного специалиста в определённой предметной области и используется для консультаций, помощи в принятии сложных решений, для решения неформализованных задач.

В данной работе необходимо реализовать экспертную систему в среде CLIPS на основе продукционной модели в области «Выбор домашнего животного на основе заданных характеристик».

Минимальный размер бинарного дерева: 4 яруса, 30 узлов.

1 Предметная область

В качестве предметной области был выбран «выбор домашнего животного на основе заданных характеристик». Данная область знаний включает в себя информацию, которая используется для подбора подходящего питомца в зависимости от образа жизни, жилищных условий и личных предпочтений владельца.

Факты, используемые для классификации, выбраны таким образом, чтобы позволить пользователю однозначно определить наиболее подходящего питомца. Ключевые характеристики для классификации: важность ежедневного общения, готовность к прогулкам и дрессировке, предпочтение активного или пассивного питомца, допустимость ночной активности, желание аквариума или птицы.

Почему была выбрана данная тема

Выбор домашнего животного является типичной неформализованной задачей: не существует единого алгоритма решения, результат зависит от многих субъективных факторов — образа жизни, жилищных условий, наличия времени и личных предпочтений. Такие задачи идеально подходят для продукционной модели экспертной системы, поскольку каждый характерный признак можно однозначно выразить в виде правила ЕСЛИ-ТО:

ЕСЛИ (важно ежедневное общение) И (есть время на прогулки)
И (подходит крупное животное)
ТО рекомендуется Собака

Тема актуальна: в Бангладеш за последние три года количество владельцев домашних животных более чем удвоилось. Объём рынка зоотоваров составляет около 2 млрд така в год с ежегодным ростом свыше 20% [1].

Как были выбраны животные

Все 16 животных, включённых в систему, выбраны на основании их популярности и доступности в Бангладеш, что подтверждается достоверными источниками:

- По данным The Daily Star (Бангладеш) [1], около 90% владельцев питомцев держат кошек — это самый популярный питомец в стране. Рынок зоотоваров растёт более чем на 20% в год.
- Согласно академическому исследованию 2024 года (ResearchGate) [2]: кошки — 56,2% владельцев, птицы — 35,5%, собаки — 11,1%, кролики — 5% от всех питомцев.
- По данным торговой площадки Vikroy.com [4] (крупнейший онлайн-рынок Бангладеш), в активной продаже представлены: собаки, кошки, кролики, попугаи, голуби, майны, канарейки, аквариумные рыбки, черепахи, хомяки, морские свинки, декоративные крысы, кореллы, волнистые попугайчики, неразлучники, золотые рыбки.
- По данным BdFISH [5], в Бангладеш насчитывается более 70 видов декоративных и аквариумных рыб, активно продающихся в зоомагазинах Дакки и Раджшахи.
- Зоорынок Катабон (Dhaka) [6] подтверждает доступность всех 16 видов животных в физических магазинах.

Таким образом, критерий отбора: широко распространённые, доступные и безопасные домашние животные в Бангладеш, пригодные для содержания в квартире или частном доме.

Список животных (факты экспертной системы)

Данный перечень представляет собой список всех возможных результатов работы экспертной системы:

1. Собака
2. Кролик
3. Кошка
4. Морская свинка
5. Хомяк
6. Декоративная крыса
7. Голубь
8. Майна
9. Аквариумные рыбки
10. Золотая рыбка
11. Черепаха
12. Корелла
13. Попугай
14. Канарейка
15. Неразлучник
16. Волнистый попугайчик

2 Математическое описание

2.1 Экспертная система и её особенности

Экспертная система — это компьютерная система, которая использует знания экспертов в определённой области для решения сложных проблем и предоставления рекомендаций или прогнозов.

Экспертные системы главным образом основаны на эвристическом поиске решения, а не на исполнении известного алгоритма. В этом одно из главных преимуществ технологии экспертных систем перед традиционным подходом к разработке программ. Именно это и позволяет им так хорошо справляться с поставленными перед ними задачами.

Экспертная система работает в двух основных режимах:

- в режиме приобретения знаний;
- в режиме решения задачи (называемом также режимом консультаций, или режимом использования экспертной системы).

В режиме приобретения знаний работу с экспертной системой осуществляет эксперт при посредничестве инженера по знаниям. В этом режиме эксперт, используя компонент приобретения знаний, наполняет систему знаниями (данными), которые, в свою очередь, позволяют системе в режиме решения уже без участия эксперта решать задачи из данной предметной области.

В режиме решения задачи (или так называемом режиме консультации) общение с экспертными системами осуществляет непосредственно конечный пользователь, которого интересует конечный итог работы и иногда способ его получения. В зависимости от назначения экспертной системы пользователь не обязательно должен быть специалистом в данной проблемной области. В этом случае он обращается к экспертным системам за результатом, не имея достаточных знаний для получения результатов.

Компоненты структуры экспертной системы:

- **Интерфейс пользователя:** обеспечивает взаимодействие между пользователем и экспертной системой. Он может быть графическим, текстовым или голосовым, позволяя пользователю задавать вопросы, получать ответы, вводить данные и взаимодействовать с системой. Интерфейс должен быть интуитивно понятным и удобным для пользователей.
- **База знаний:** знания — это правила, законы, закономерности, полученные в результате профессиональной деятельности в пределах предметной области. База знаний — база данных, содержащая правила вывода и информацию о человеческом опыте и знаниях в некоторой предметной области. Другими словами, это набор таких закономерностей, которые устанавливают связи между вводимой и выводимой информацией.
- **Данные:** данные — это совокупность фактов и идей, представленных в формализованном виде. На данных основываются закономерности для предсказания, прогнозирования. Продвинутое интеллектуальные системы способны учиться на основе этих данных, добавляя новые знания в базу знаний.
- **Машина логического вывода:** является программным компонентом, который определяет антецеденты правил (если таковые имеются), которые выполняются согласно фактам. Для этого машина логического вывода выполняет следующие действия:

– выбирает правила, которым соответствуют факты;

- распределяет выбранные правила по приоритетам;
- выполняет правило с высшим приоритетом.
- **Интеллектуальный редактор:** обеспечивает режим пополнения базы знаний. Позволяет эксперту при посредничестве инженера по знаниям вводить новые правила и факты в систему.

2.2 Продукционная модель представления знаний

По своей сути продукционные модели знаний близки к логическим моделям, что позволяет организовать весьма эффективные процедуры логического вывода данных.

Традиционная продукционная модель знаний включает в себя следующие базовые компоненты:

1. набор правил (или продукций), представляющих базу знаний продукционной системы;
 2. рабочую память, в которой хранятся исходные факты, а также факты, выведенные из исходных фактов при помощи механизма логического вывода;
 3. сам механизм логического вывода, позволяющий из имеющихся фактов, согласно имеющимся правилам вывода, выводить новые факты.
- В основе продукционной модели представления знаний находится конструктивная часть — продукция (правило):

$$\text{IF } \langle \text{условие} \rangle, \text{ THEN } \langle \text{действие } \text{№}1 \rangle, \text{ ELSE } \langle \text{действие } \text{№}2 \rangle$$
 - Продукция состоит из двух частей: условие — антецедент, действие — консеквент.
 - Условия можно сочетать с помощью логических функций AND, OR.
 - Антецеденты и консеквенты составленных правил формируются из атрибутов и значений.
 - В базе данных продукционной системы хранятся правила, истинность которых установлена заранее при решении определённой задачи. Правило срабатывает, если при сопоставлении фактов, содержащихся в базе данных с антецедентом правила, которое подвергается проверке, имеет место совпадение. Результат работы правила заносится в базу данных.

2.3 Бинарное дерево решений

Экспертная система представлена бинарным деревом решений — ациклическим орграфом с причинно-следственной связью. В дереве решений вопросы — это узлы решения, а конечные результаты — это листья.

В итоге работа экспертной системы означает путь по графу. Такой путь состоит из последовательности шагов, на каждом из которых пользователь должен решить, по какой дуге он пойдёт из очередной вершины. При этом все вопросы являются однозначными (да/нет).

Описание дерева решений:

- **Узлы (Nodes):** узлы представляют состояния или решения в дереве решений. Каждый узел имеет определённые свойства, которые могут включать условия, переменные или факторы, по которым принимается решение.

- **Рёбра (Edges):** рёбра представляют переходы или ветвления между узлами. Они определяют логическую последовательность принятия решений и связи между различными состояниями. В дереве они представлены двух видов, где пунктирное ребро означает ответ «нет», а сплошное — ответ «да».
- **Условия (Conditions):** условия определяют логические выражения или предикаты, которые должны быть истинными для перехода по определённому ребру. Они могут быть основаны на значениях переменных или факторов, которые определяются в экспертной системе.
- **Правила (Rules):** правила являются частью узлов и определяют логику принятия решений в конкретных состояниях. Они могут быть представлены в виде условий-действий или логических правил, которые определяют, какие действия или выводы следует сделать на основе заданных условий.
- **Выводы (Conclusions):** выводы представляют собой результаты или решения, которые могут быть получены в конечном узле или листе дерева решений. Они могут быть представлены в виде конкретных значений, классификаций или действий, которые следует принять на основе принятых решений.

Числовые характеристики дерева: количество узлов — 31, количество ярусов — 4, количество фактов (листьев) — 16, количество правил — 15.

Сформированное в ходе выполнения курсовой работы дерево представлено в виде диаграммы на рис. 1.

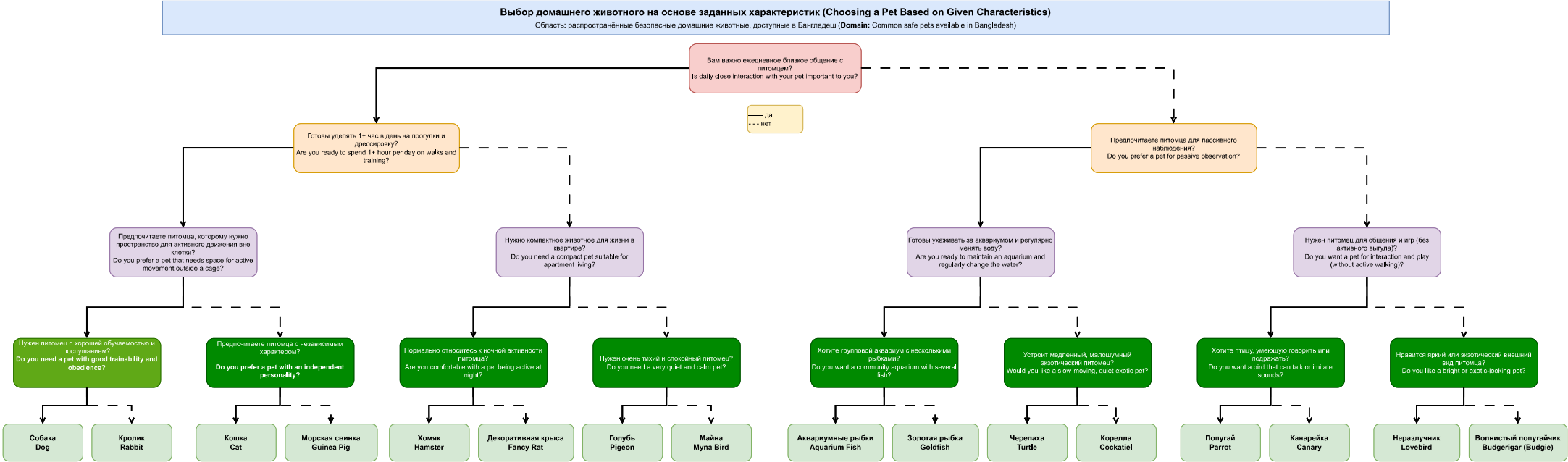


Рисунок 1 — Бинарное дерево решений

3 Особенности реализации

Экспертная система реализована в среде CLIPS (C Language Integrated Production System) на основе продукционной модели. База знаний системы представлена набором из 31 правила (`defrule`): 15 правил реализуют узлы дерева решений и 16 правил — листовые узлы с рекомендациями. Рабочая память CLIPS хранит текущие факты о предпочтениях пользователя. Машина вывода автоматически выбирает и активирует подходящие правила. В данном разделе описаны ключевые функции и правила программы [10, 8, 9].

3.1 Общая функция задания вопроса и проверки ввода

Функция `ask-question` является универсальной функцией запроса ввода у пользователя с проверкой корректности. Принимает текст вопроса `?question` и список допустимых значений `?$allowed-values`. Выводит вопрос, считывает ответ функцией `read`, преобразует его к нижнему регистру функцией `lowcase`. Если ответ не входит в список допустимых значений (`member$`), выводит сообщение об ошибке и повторяет запрос в цикле `while`. Возвращает принятый ответ. Код представлен в листинге 1.

```
1 (deffunction ask-question (?question $?allowed-values)
2   (printout t ?question)
3   (bind ?answer (read))
4   (if (lexemep ?answer)
5       then (bind ?answer (lowcase ?answer)))
6   (while (not (member$ ?answer ?allowed-values)) do
7     (printout t " Invalid input. Please enter: " ?allowed-values crlf)
8     (printout t ?question)
9     (bind ?answer (read))
10    (if (lexemep ?answer)
11        then (bind ?answer (lowcase ?answer))))
12   ?answer)
```

Листинг 1. Функция `ask-question`

3.2 Функция для вопросов типа да/нет

Функция `yesno` является обёрткой над `ask-question`. Принимает только текст вопроса и вызывает `ask-question` с фиксированным списком допустимых значений (`yes no y n`). Нормализует ответ: если пользователь ввёл `y` или `yes` — возвращает символ `yes`, иначе — `no`. Это обеспечивает единообразие: все последующие правила системы работают только с символами `yes` и `no`, не зависимо от того, что именно ввёл пользователь. Код представлен в листинге 2.

```
1 (deffunction yesno (?question)
2   (bind ?response (ask-question ?question yes no y n))
3   (if (or (eq ?response yes) (eq ?response y))
4       then yes
5       else no))
```

Листинг 2. Функция `yesno`

3.3 Правило вывода рекомендации

Правило `print-rec` имеет наивысший приоритет (`salience 1000`), что гарантирует его срабатывание раньше любого другого правила в момент появления в рабочей памяти факта (`rec ...`). Правило выводит итоговую рекомендацию на экран, изымает факт

(rec ...) из рабочей памяти и добавляет факт (restart), запуская тем самым правило перезапуска. Код представлен в листинге 3.

```
1 (defrule print-rec
2   (declare (salience 1000))
3   ?rec <- (rec ?item)
4   =>
5   (printout t crlf "Recommended pet: " ?item crlf crlf)
6   (retract ?rec)
7   (assert (restart)))
```

Листинг 3. Правило print-rec

3.4 Правило перезапуска

Правило restart-system срабатывает при появлении факта (restart) в рабочей памяти. Изымает факт (restart), запрашивает у пользователя желание получить новую рекомендацию через функцию ask-question с валидацией ввода. При положительном ответе (yes или y) выводит заголовок программы и помещает в рабочую память факт с ответом на первый вопрос дерева, запуская новый сеанс. При отрицательном — выводит сообщение Goodbye! и останавливает машину вывода командой halt. Код представлен в листинге 4.

```
1 (defrule restart-system
2   ?r <- (restart)
3   =>
4   (retract ?r)
5   (bind ?a (ask-question "Would you like a new recommendation? (yes/no): "
6                           yes no y n))
7   (if (or (eq ?a yes) (eq ?a y))
8       then
9         (printout t crlf
10          "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
11         (assert (interaction
12                  (yesno "Is daily close interaction with your pet important to you? (yes/no): ")))
13       else
14         (printout t crlf "Goodbye!" crlf)
15         (halt)))
```

Листинг 4. Правило restart-system

3.5 Правила дерева решений

Каждый узел дерева реализован парой правил — для ответа «да» и для ответа «нет». Каждое правило: связывает совпавший факт переменной ?f, изымает его из рабочей памяти (retract), задаёт следующий вопрос через yesno и добавляет новый факт (assert). Изъятие факта после срабатывания правила обеспечивает чистоту рабочей памяти: по завершении сеанса она полностью пуста, что гарантирует корректную работу при перезапуске. Стартовое правило first срабатывает только при полностью пустой рабочей памяти и выводит заголовок программы. В листинге 5 приведён фрагмент, демонстрирующий стартовое правило и правила первых двух уровней дерева; полная реализация представлена в Приложении A1.

```
1 ; Entry point - fires only when working memory is empty
2 (defrule first ""
3   (not (interaction ?))
```

```

4      (not (rec ?))
5      (not (restart))
6      =>
7      (printout t "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
8      (assert (interaction
9          (yesno "Is daily close interaction with your pet important to you? (yes/no): "))))
10
11 ; Level 2 - n2 (yes branch)
12 (defrule interaction-yes ""
13     ?f <- (interaction yes) (not (rec ?)) =>
14     (retract ?f)
15     (assert (walks
16         (yesno "Are you ready to spend 1+ hour per day on walks and training? (yes/no): ")))
17 )
18 ; Level 2 - n3 (no branch)
19 (defrule interaction-no ""
20     ?f <- (interaction no) (not (rec ?)) =>
21     (retract ?f)
22     (assert (passive
23         (yesno "Do you prefer a pet for passive observation? (yes/no): "))))

```

Листинг 5. Фрагмент правил дерева решений (стартовое правило и первые два уровня)

3.6 Правила-листья (рекомендации)

16 правил-листьев реализуют листовые узлы дерева решений. Каждое правило проверяет наличие факта четвёртого уровня дерева и отсутствие факта (rec ...). При срабатывании изымает свой факт и добавляет (rec "Название животного"). Это немедленно активирует правило print-rec с приоритетом 1000. Код представлен в листинге 6.

```

1 (defrule trainable-yes "" ?f <- (trainable yes) (not (rec ?)) =>
2     (retract ?f) (assert (rec "Dog")))
3 (defrule trainable-no "" ?f <- (trainable no) (not (rec ?)) =>
4     (retract ?f) (assert (rec "Rabbit")))
5
6 (defrule independent-yes "" ?f <- (independent yes) (not (rec ?)) =>
7     (retract ?f) (assert (rec "Cat")))
8 (defrule independent-no "" ?f <- (independent no) (not (rec ?)) =>
9     (retract ?f) (assert (rec "Guinea Pig")))
10
11 ; ... (remaining 12 leaf rules follow the same pattern)
12
13 (defrule bright-yes "" ?f <- (bright yes) (not (rec ?)) =>
14     (retract ?f) (assert (rec "Lovebird")))
15 (defrule bright-no "" ?f <- (bright no) (not (rec ?)) =>
16     (retract ?f) (assert (rec "Budgerigar")))

```

Листинг 6. Правила-листья (фрагмент)

4 Результаты работы программы

Результатом работы является готовая программа, реализованная в среде CLIPS. Запуск производится командами `(load "pet_expert.clp")`, `(reset)`, `(run)` в интерпретаторе CLIPS. После запуска выводится заголовок и первый вопрос, находящийся в корне бинарного дерева. Пользователь отвечает **yes** (или **y**) для ответа «да» и **no** (или **n**) для ответа «нет». Программа проверяет корректность ввода: при неверном вводе выдаётся сообщение об ошибке и вопрос задаётся повторно. После прохождения четырёх уровней дерева система выводит рекомендацию и предлагает начать новый сеанс. Ниже представлены примеры работы программы.

4.1 Пример №1: путь yes-yes-yes-yes, рекомендация — Dog

На рис. 2 показан первый сеанс работы программы. Пользователь отвечает **y** на все четыре вопроса (путь да-да-да-да): система рекомендует *Dog* (Собака). Данный путь соответствует крайней левой ветви бинарного дерева решений: $n1 \rightarrow n2 \rightarrow n4 \rightarrow n8 \rightarrow \text{Dog}$. После вывода рекомендации программа предлагает начать новый сеанс.

```
CLIPS> (reset)
CLIPS> (run)
=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): y
Are you ready to spend 1+ hour per day on walks and training? (yes/no): y
Do you prefer a pet that needs space for active movement outside a cage? (yes/no): y
Do you need a pet with good trainability and obedience? (yes/no): y

Recommended pet: Dog

Would you like a new recommendation? (yes/no): █
```

Рис. 2. Пример №1: путь yes-yes-yes-yes, рекомендация — Dog

4.2 Пример №2: повторный сеанс и завершение работы

На рис. 3 представлен повторный сеанс с другими ответами. Программа демонстрирует корректную работу механизма перезапуска: рабочая память полностью очищается между сеансами с помощью операции **retract** в каждом правиле, и система задаёт вопросы заново с первого уровня дерева. По завершении сеанса пользователь отказывается от нового сеанса и программа выводит **Goodbye!** и останавливается командой **halt**.

```

Would you like a new recommendation? (yes/no): y

=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): n
Do you prefer a pet for passive observation? (yes/no): n
Do you want a pet for interaction and play without active walking? (yes/no): y
Do you want a bird that can talk or imitate sounds? (yes/no): n

Recommended pet: Canary

Would you like a new recommendation? (yes/no): n

Goodbye!
CLIPS>

```

Рис. 3. Пример №2: повторный сеанс с новыми ответами и завершение работы

4.3 Пример №3: обработка некорректного ввода

На рис. 4 показана обработка некорректного ввода. Пользователь вводит произвольную строку `asdf` вместо допустимого ответа. Программа выводит сообщение об ошибке с указанием допустимых значений (`yes no y n`) и повторяет вопрос. Сеанс продолжается только после получения корректного ответа. Данный механизм реализован в функции `ask-question` с помощью цикла `while`: программа не завершается и не переходит к следующему вопросу до тех пор, пока не получит допустимый ввод.

```

=== Expert System: Pet Selection (Bangladesh) ===

Is daily close interaction with your pet important to you? (yes/no): n
Do you prefer a pet for passive observation? (yes/no): y
Are you ready to maintain an aquarium and regularly change the water? (yes/no): asdf
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no): 3
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no): 5
  Invalid input. Please enter: (yes no y n)
Are you ready to maintain an aquarium and regularly change the water? (yes/no):
y
Do you want a community aquarium with several fish? (yes/no): y

Recommended pet: Aquarium Fish

Would you like a new recommendation? (yes/no):

```

Рис. 4. Пример №3: обработка некорректного ввода — система выводит сообщение об ошибке и повторяет вопрос

Заключение

В результате курсовой работы была реализована экспертная система в среде CLIPS по теме «Выбор домашнего животного на основе заданных характеристик (Бангладеш)» [1, 2, 3], представленная бинарным деревом решений со следующими числовыми характеристиками: количество узлов — 31, количество ярусов — 4, количество фактов — 16, количество правил — 15. Знания реализованной системы представлены в виде продукционной модели: база знаний состоит из 31 правила, рабочая память хранит текущие факты сеанса, машина вывода CLIPS автоматически управляет активацией правил. Данная система позволяет пользователю однозначно определить наиболее подходящего питомца, исходя из его образа жизни, жилищных условий и личных предпочтений.

Достоинства

Достоинства использования CLIPS и продукционной модели:

- Знания (правила) отделены от механизма управления (машины вывода CLIPS): добавление нового узла в дерево требует только добавления новой пары правил, без изменения логики управления.
- Алгоритм обхода бинарного дерева эффективен: для получения рекомендации требуется ровно 4 вопроса в соответствии с числом ярусов дерева.
- Механизм приоритетов (*saliency*) обеспечивает предсказуемый порядок срабатывания правил: правило вывода рекомендации всегда активируется первым.

Недостатки

- При необходимости добавления или удаления узла необходимо перестроить дерево решений и добавить соответствующие правила, следовательно, система не является полностью гибкой.
- Экспертная система подходит для задач с ограниченным количеством факторов и простыми взаимодействиями; для более сложных алгоритмов требуются более продвинутые методы — семантические сети и фреймовые модели [9].

Масштабирование

Программа может быть расширена путём добавления новых вопросов и животных, уточнения критериев классификации или расширения бинарного дерева решений. Другой способ масштабирования — переход к более сложным структурам представления знаний в CLIPS: использование `deftemplate` для структурированных фактов или подключение внешней базы знаний [8].

Список использованной литературы

- [1] "Market for pet food, accessories growing- / The Daily Star. — URL: <https://www.thedailystar.net/business/economy/news/market-pet-food-accessories-growing-3422306> (Дата обращения: 11.05.2026)
- [2] The evolving paradigm of pet animal rearing in Bangladesh / ResearchGate. — URL: <https://www.researchgate.net/publication/388758458> (Дата обращения: 11.05.2026)
- [3] Navigating Pet Welfare in Bangladesh: A Multilayered Analysis of Ownership, Ethics and Service Accessibility / ResearchGate. — URL: <https://www.researchgate.net/publication/398122556> (Дата обращения: 11.05.2026)
- [4] Animals & Pets for sale in Bangladesh / Bikroy.com. — URL: <https://bikroy.com/en/ads/bangladesh/pets-animals> (Дата обращения: 11.05.2026)
- [5] Aquarium Fishes of Bangladesh: Part-1 / BdFISH Feature. — URL: <https://en.bdfish.org/2011/01/aquarium-fish-bangladesh-1/> (Дата обращения: 11.05.2026)
- [6] Katabon Online — Best Quality Pet Supplies (Dhaka pet market). — URL: <https://katabononline.com/> (Дата обращения: 11.05.2026)
- [7] The Rising Trend of Pet Ownership in Bangladesh / Daily Business Eye. — URL: <https://www.dailybusinessseye.com/todayspaper/paper/4183> (Дата обращения: 11.05.2026)
- [8] Хабаров С.П. CLIPS — язык построения экспертных систем. — Санкт-Петербург, 2013. — 80 с.
- [9] Гаврилова Т.А., Частиков А.П. Разработка экспертных систем. Среда CLIPS. — Санкт-Петербург: БХВ-Петербург, 2003. — 393 с.
- [10] Секция «Телематика» / текст: электронный. — URL: <https://tema.spbstu.ru/tgraph/> (Дата обращения: 11.05.2026)

Приложение А. Файл pet_expert.clp

```
1 ; =====
2 ; Expert System: Choosing a Pet (Bangladesh)
3 ; Binary decision tree -- 4 levels, 15 decision nodes, 16 recommendations
4 ; -----
5 ; Usage: CLIPS> (load "pet_expert.clp")
6 ;         CLIPS> (reset)
7 ;         CLIPS> (run)
8 ; Input: yes / y for YES
9 ;        no / n for NO
10 ; =====
11
12
13 ; -----
14 ; 3.1 General question function
15 ;     Asks ?question, reads answer, loops until it matches
16 ;     one of $?allowed-values (case-insensitive).
17 ; -----
18 (deffunction ask-question (?question $?allowed-values)
19   (printout t ?question)
20   (bind ?answer (read))
21   (if (lexemep ?answer)
22       then (bind ?answer (lowercase ?answer)))
23   (while (not (member$ ?answer ?allowed-values)) do
24     (printout t " Invalid input. Please enter: " ?allowed-values crlf)
25     (printout t ?question)
26     (bind ?answer (read))
27     (if (lexemep ?answer)
28         then (bind ?answer (lowercase ?answer))))
29   ?answer)
30
31
32 ; -----
33 ; 3.2 Yes/No wrapper
34 ;     Calls ask-question with allowed values yes/no/y/n.
35 ;     Returns the symbol yes or no.
36 ; -----
37 (deffunction yesno (?question)
38   (bind ?response (ask-question ?question yes no y n))
39   (if (or (eq ?response yes) (eq ?response y))
40       then yes
41       else no))
42
43
44 ; -----
45 ; 3.3 Restart rule
46 ;     Fires when (restart) fact appears.
47 ;     Asks the user whether to run again.
48 ;     If yes -- starts a new session by asserting n1 answer.
49 ;     If no -- prints goodbye and halts.
50 ; -----
51 (defrule restart-system
52   ?r <- (restart)
53   =>
54   (retract ?r)
55   (bind ?a (ask-question "Would you like a new recommendation? (yes/no): " yes no y n))
56   (if (or (eq ?a yes) (eq ?a y))
57       then
58         (printout t crlf "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
```

```

59         (assert (interaction
60             (yesno "Is daily close interaction with your pet important to you? (yes/no):
        )))
61     else
62         (printout t crlf "Goodbye!" crlf)
63         (halt)))
64
65
66 ; -----
67 ; 3.4 Print recommendation
68 ;     Highest salience so it fires before any other rule.
69 ;     Prints the result, retracts it, and triggers restart.
70 ; -----
71 (defrule print-rec
72     (declare (salience 1000))
73     ?rec <- (rec ?item)
74     =>
75     (printout t crlf "Recommended pet: " ?item crlf crlf)
76     (retract ?rec)
77     (assert (restart)))
78
79
80 ; =====
81 ; 3.5 Question rules (one rule per decision node)
82 ;     Each rule binds its matching fact with ?f and retracts
83 ;     it immediately -- so working memory is fully clean after
84 ;     every session and restart works without leftover facts.
85 ;     Solid edge = YES answer
86 ;     Dashed edge = NO answer
87 ; =====
88
89 ; --- Entry point -- Level 1 -- n1 -----
90 (defrule first ""
91     (not (interaction ?))
92     (not (rec ?))
93     (not (restart))
94     =>
95     (printout t "=== Expert System: Pet Selection (Bangladesh) ===" crlf crlf)
96     (assert (interaction
97         (yesno "Is daily close interaction with your pet important to you? (yes/no): "))))
98
99 ; --- Level 2 -- n2 / n3 -----
100 (defrule interaction-yes ""
101     ?f <- (interaction yes) (not (rec ?)) =>
102     (retract ?f)
103     (assert (walks
104         (yesno "Are you ready to spend 1+ hour per day on walks and training? (yes/no): ")))
105 )
106
107 (defrule interaction-no ""
108     ?f <- (interaction no) (not (rec ?)) =>
109     (retract ?f)
110     (assert (passive
111         (yesno "Do you prefer a pet for passive observation? (yes/no): "))))
112
113 ; --- Level 3 -- n4 / n5 / n6 / n7 -----
114 (defrule walks-yes ""
115     ?f <- (walks yes) (not (rec ?)) =>
116     (retract ?f)
117     (assert (space

```

```

117         (yesno "Do you prefer a pet that needs space for active movement outside a cage? (
118             yes/no): "))))
119 (defrule walks-no ""
120     ?f <- (walks no) (not (rec ?)) =>
121     (retract ?f)
122     (assert (compact
123         (yesno "Do you need a compact pet suitable for apartment living? (yes/no): "))))
124
125 (defrule passive-yes ""
126     ?f <- (passive yes) (not (rec ?)) =>
127     (retract ?f)
128     (assert (aquarium
129         (yesno "Are you ready to maintain an aquarium and regularly change the water? (yes/
130             no): "))))
131
132 (defrule passive-no ""
133     ?f <- (passive no) (not (rec ?)) =>
134     (retract ?f)
135     (assert (playpet
136         (yesno "Do you want a pet for interaction and play without active walking? (yes/no):
137             "))))
138
139 ; --- Level 4 -- n8 / n9 / n10 / n11 / n12 / n13 / n14 / n15
140 (defrule space-yes ""
141     ?f <- (space yes) (not (rec ?)) =>
142     (retract ?f)
143     (assert (trainable
144         (yesno "Do you need a pet with good trainability and obedience? (yes/no): "))))
145
146 (defrule space-no ""
147     ?f <- (space no) (not (rec ?)) =>
148     (retract ?f)
149     (assert (independent
150         (yesno "Do you prefer a pet with an independent personality? (yes/no): "))))
151
152 (defrule compact-yes ""
153     ?f <- (compact yes) (not (rec ?)) =>
154     (retract ?f)
155     (assert (nocturnal
156         (yesno "Are you comfortable with a pet being active at night? (yes/no): "))))
157
158 (defrule compact-no ""
159     ?f <- (compact no) (not (rec ?)) =>
160     (retract ?f)
161     (assert (quiet
162         (yesno "Do you need a very quiet and calm pet? (yes/no): "))))
163
164 (defrule aquarium-yes ""
165     ?f <- (aquarium yes) (not (rec ?)) =>
166     (retract ?f)
167     (assert (community
168         (yesno "Do you want a community aquarium with several fish? (yes/no): "))))
169
170 (defrule aquarium-no ""
171     ?f <- (aquarium no) (not (rec ?)) =>
172     (retract ?f)
173     (assert (slowexotic
174         (yesno "Would you like a slow-moving, quiet exotic pet? (yes/no): "))))

```

```

174 (defrule playpet-yes ""
175     ?f <- (playpet yes) (not (rec ?)) =>
176     (retract ?f)
177     (assert (talking
178         (yesno "Do you want a bird that can talk or imitate sounds? (yes/no): "))))
179
180 (defrule playpet-no ""
181     ?f <- (playpet no) (not (rec ?)) =>
182     (retract ?f)
183     (assert (bright
184         (yesno "Do you like a bright or exotic-looking pet? (yes/no): "))))
185
186
187 ; =====
188 ; 3.6 Recommendation rules (one rule per leaf node)
189 ;     Each also retracts its fact so working memory is
190 ;     completely empty when the recommendation is made.
191 ; =====
192
193 (defrule trainable-yes "" ?f <- (trainable yes) (not (rec ?)) => (retract ?f) (assert (
194     rec "Dog")))
195
196 (defrule trainable-no "" ?f <- (trainable no) (not (rec ?)) => (retract ?f) (assert (
197     rec "Rabbit")))
198
199 (defrule independent-yes "" ?f <- (independent yes) (not (rec ?)) => (retract ?f) (assert (
200     rec "Cat")))
201
202 (defrule independent-no "" ?f <- (independent no) (not (rec ?)) => (retract ?f) (assert (
203     rec "Guinea Pig")))
204
205 (defrule nocturnal-yes "" ?f <- (nocturnal yes) (not (rec ?)) => (retract ?f) (assert (
206     rec "Hamster")))
207
208 (defrule nocturnal-no "" ?f <- (nocturnal no) (not (rec ?)) => (retract ?f) (assert (
209     rec "Fancy Rat")))
210
211 (defrule quiet-yes "" ?f <- (quiet yes) (not (rec ?)) => (retract ?f) (assert (
212     rec "Pigeon")))
213
214 (defrule quiet-no "" ?f <- (quiet no) (not (rec ?)) => (retract ?f) (assert (
215     rec "Myna Bird")))
216
217 (defrule community-yes "" ?f <- (community yes) (not (rec ?)) => (retract ?f) (assert (
218     rec "Aquarium Fish")))
219
220 (defrule community-no "" ?f <- (community no) (not (rec ?)) => (retract ?f) (assert (
221     rec "Goldfish")))
222
223 (defrule slowexotic-yes "" ?f <- (slowexotic yes) (not (rec ?)) => (retract ?f) (assert (
224     rec "Turtle")))
225
226 (defrule slowexotic-no "" ?f <- (slowexotic no) (not (rec ?)) => (retract ?f) (assert (
227     rec "Cockatiel")))
228
229 (defrule talking-yes "" ?f <- (talking yes) (not (rec ?)) => (retract ?f) (assert (
230     rec "Parrot")))
231
232 (defrule talking-no "" ?f <- (talking no) (not (rec ?)) => (retract ?f) (assert (
233     rec "Canary")))
234
235 (defrule bright-yes "" ?f <- (bright yes) (not (rec ?)) => (retract ?f) (assert (
236     rec "Lovebird")))
237
238 (defrule bright-no "" ?f <- (bright no) (not (rec ?)) => (retract ?f) (assert (
239     rec "Budgerigar")))

```